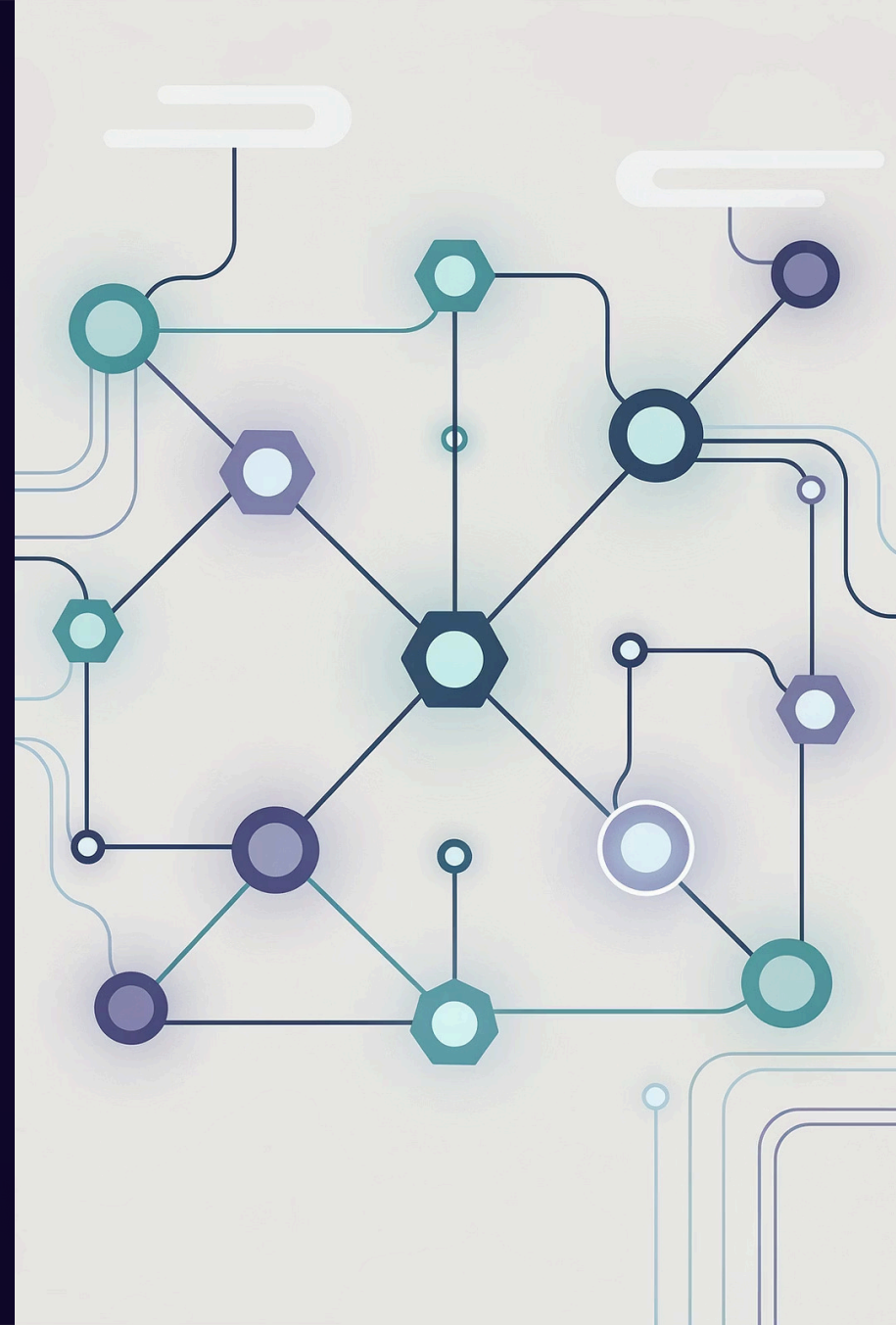


Mastering the Model Context Protocol (MCP)

Tools, Resources, and Advanced Prompting for AI Agent Systems



Before We Start

A session designed to sharpen how you think about **tools, resources, prompts, and integrations** in MCP-aware agent systems.

Data Access

When you think about how your agents access real data today, what comes to mind first: direct database queries, REST APIs, or something else entirely?

Mental Model

How clearly do you distinguish between what a prompt does, what a tool does, and what a static resource does in your current mental model?

Error Handling

When external services fail or misbehave, how predictable is your current error handling from a user's point of view?

📌 It is normal if these areas feel fuzzy — this session is designed to sharpen how you think about tools, resources, prompts, and integrations.

What Is a Tool, and How Does It Differ from Prompts and Resources?

In the context of an MCP-aware agent, understanding the boundaries between these three concepts is foundational.

What Is a Tool?

A tool is a **structured capability** the agent can invoke to perform an action or retrieve fresh data from outside the model.

- Maps to operations like "get order status", "check stock", or "create booking"
- Has clear, defined inputs and outputs
- Executes new business logic on demand

Prompts vs. Resources

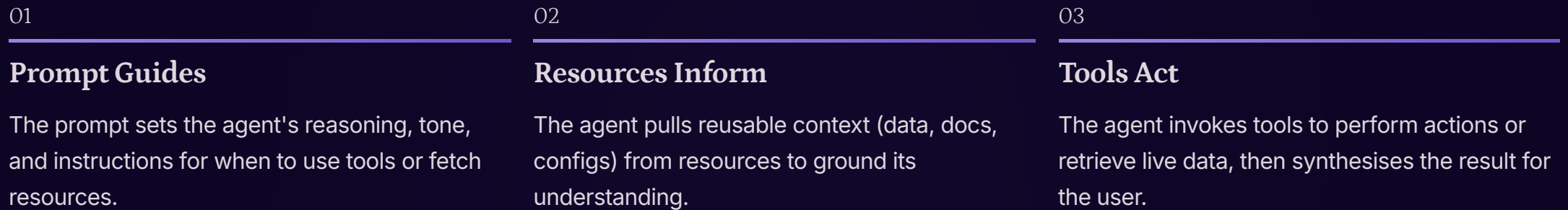
Prompts describe how the agent should think and speak — they do not themselves fetch data or perform actions.

Resources represent reusable information or data sources that can be fetched or subscribed to without executing new business logic.

 Think of it as: prompts = reasoning, tools = actions, resources = reusable context.

How Do Prompts, Tools, and Resources Work Together?

A single user request can involve all three components working in concert. Here is what that flow looks like in practice.



Seeing all three roles in action makes the collaboration concrete — the prompt guides, resources inform, and tools act.

Why Separate These Roles?

- Separation makes it easier to evolve each piece independently
- Prompts handle reasoning, tools handle actions, resources handle reusable context

Practical Benefits

- Clarifies where to apply caching, security, or versioning
- Avoids a single monolithic prompt that is hard to maintain or debug

What Is MCP, and Why Does It Matter?

MCP in Simple Terms

The **Model Context Protocol (MCP)** is an open standard that lets language-model applications talk to external tools, resources, and prompts using a consistent message format and lifecycle.

It defines how clients and servers connect, describe capabilities, make calls, and stream results — independent of the underlying business logic.

Why a Standard Matters

No Re-invention

Avoids custom protocols for every service

Portability

Tool integrations work across projects and teams

Uniform Clients

Agents use different MCP servers through the same client machinery



What Does FastMCP Add on Top of MCP?

FastMCP is a developer-focused layer that removes friction when building MCP-compliant servers in Python.



Minimal Boilerplate

FastMCP handles low-level protocol and lifecycle work, letting you write Python functions and expose them as MCP tools without ceremony.



Built-in Structure

Provides structure around registration, schema, validation, streaming, and error handling so you can focus on your business logic.



Familiar Patterns

You work in familiar Python patterns while still producing a standards-compliant MCP server that clients can discover and invoke.



Easier Debugging

Upfront structure from FastMCP makes your tools easier for clients to discover, invoke, and debug across environments.

What Is the Basic Protocol Flow Between MCP Client and Server?

1. Establish Connection

Client connects to the MCP server over a supported transport such as HTTP, SSE, or stdio.

2. Discover Capabilities

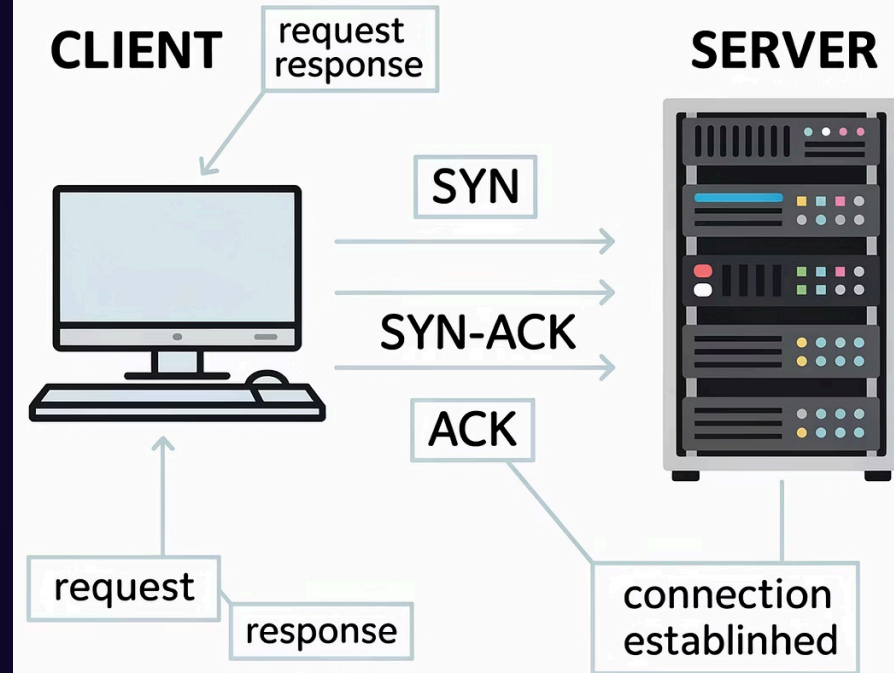
Client lists available tools, resources, and prompts so it knows what it can use during the conversation.

3. Send Call Request

Client sends the selected tool name and structured parameters to the MCP server.

4. Receive Result

Server executes business logic and returns a structured result or error, which the caller interprets and uses in its reasoning.



How Do Transports Fit In Without Changing the Protocol?

Available Transports

MCP clients and servers can communicate over multiple transports, each suited to different deployment environments and latency constraints.

- **stdio** — convenient for local processes and CLI tools
- **HTTP(S)** — natural for networked and cloud-hosted services
- **Server-Sent Events (SSE)** — suited for streaming and real-time updates

What Stays the Same?

The **MCP message format and lifecycle** are consistent regardless of transport, so clients can treat servers uniformly.

- ✓ You can switch transports for deployment reasons without rewriting your higher-level integration logic.

This transport-agnostic design is one of MCP's most practical advantages — it decouples *how* you communicate from *what* you communicate.

How Do You Design Good Tools for Order and Inventory Scenarios?

Order Management Tools

`get_order_status`

Input: order ID → Output: current status and timeline

`list_recent_orders`

Input: customer ID → Output: paginated order list

`cancel_order`

Input: order ID → Output: confirmation or rejection

`fetch_invoice_details`

Input: invoice ID → Output: line items and totals

Inventory & Support Tools

`check_stock`

Input: product ID → Output: quantity available

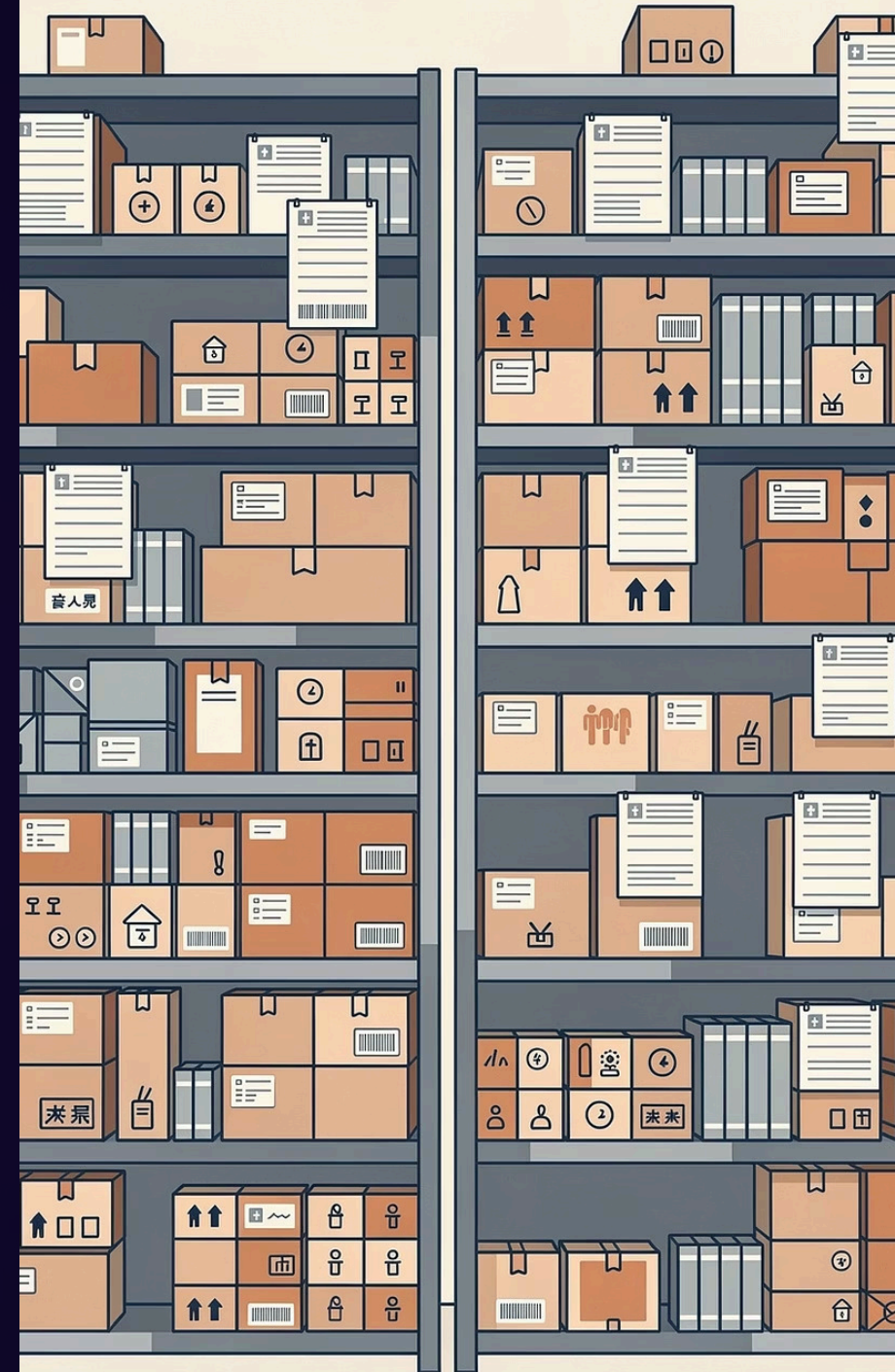
`list_available_variants`

Input: product ID → Output: sizes, colours, SKUs

`lookup_product_details`

Input: product ID → Output: specs and description

i Keep each tool focused and, where possible, idempotent — this makes behaviour easier to understand, test, and recover from.



How Can Prompts Encourage Correct Tool Use?

Well-crafted prompts are the bridge between user intent and reliable tool invocation. They shape when and how agents reach for tools.

1 Describe Tool Capabilities Explicitly

Prompts can explicitly describe tool capabilities and instruct the agent to call tools whenever the answer depends on live or authoritative data — preventing hallucination by design.

2 Guard Against Fabrication

Prompts can tell the agent to avoid fabricating answers when tools indicate that data is missing or an ID is invalid, keeping responses grounded in real system state.

3 Guide Response Presentation

Prompts can ask agents to summarise tool responses into clear, user-friendly language instead of dumping raw fields — turning structured data into helpful answers.

4 Build User Trust

Prompts can encourage agents to mention when they relied on live system data, which signals transparency and builds confidence in the agent's responses.

Error Handling and Security Considerations for Tool Calls

Common Failure Modes

Tools can fail in predictable ways — and your design should account for all of them.

Timeouts

Upstream service takes too long to respond

404 / 500 Errors

Resource not found or server-side failure

Malformed Responses

Unexpected schema or missing fields

Robust designs apply **limited retries**, return user-friendly fallbacks, and log failures distinctly from normal responses.

Security & Tool Poisoning

If attackers can influence tool inputs or backing data, they may steer agents toward harmful or incorrect outcomes.

⊗ Tools must validate inputs and enforce least privilege. Treat every tool as part of your overall attack surface.

- Validate all inputs before executing business logic
- Enforce least-privilege access on every tool
- Be aware of prompt injection vectors through tool responses
- Detailed guardrail patterns belong in a dedicated security session — but awareness starts now